

SOLID Principle

Tuesday, February 4, 2025 5:11 PM

- The SOLID principles are a set of design principles in object-oriented programming that help developers create more maintainable, understandable, and flexible software.
- The SOLID principle helps in reducing tight coupling.
- Loosely coupled classes minimize changes in your code, helps in making code more reusable, maintainable, flexible and stable.

Brief of SOLID

- **Single Responsibility Principle (SRP):** A class should have only one reason to change, which means every class should have a single responsibility or single job or single purpose.
- **Open/Closed Principle (OCP):** Software entities (classes, modules, functions, etc.) should be open for extension but closed for modification. This means you should be able to add new functionality without changing existing code.
- **Liskov's Substitution Principle (LSP):** Objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program.
 - In simpler terms, you should be able to substitute a subclass object wherever a superclass object is expected, and the program should continue to work correctly without issues.
 - LSP Example: If you have a superclass Bird with a method fly(), and a subclass Penguin that cannot fly, then Penguin should not inherit from Bird.

```
public class Bird
{
    public virtual void Fly()
    {
        Console.WriteLine("Flying");
    }
}

public class Penguin : Bird
{
    public override void Fly()
    {
        throw new InvalidOperationException("Penguins can't fly!");
    }
}
```

Fixing the Violation:

```
public class Bird
{
    public void Eat()
    {
        Console.WriteLine("Eating");
    }
}
```

```

}

// Interface for flying behavior
public interface IFlyable
{
    void Fly();
}

public class Sparrow : Bird, IFlyable
{
    public void Fly()
    {
        Console.WriteLine("Flying");
    }
}

public class Penguin : Bird
{
    public void Swim()
    {
        Console.WriteLine("Swimming");
    }
}

```

- **Interface Segregation Principle (ISP):** Definition: No client should be forced to depend on methods it does not use. This means creating specific interfaces rather than a general-purpose interface.
 - ISP) Example: Suggests that instead of having one large menu card with all items (vegetarian, non-vegetarian, drinks, sweets), you should have separate menu cards for different types of customers (e.g., a vegetarian menu, a non-vegetarian menu, a drinks menu, and a sweets menu). This way, customers only get the menu relevant to their preferences, reducing unnecessary information and making it easier to manage changes.
IVegetarianMenu , INon-VegetarianMenu, IDrinkMenu
- **Dependency Inversion Principle (DIP):** High-level code (like the main logic of your application) should not depend directly on low-level code (like specific details of how things are done).
 - Instead, both should depend on abstractions (like interfaces or abstract classes).
 - This way, you can easily change the low-level details without affecting the high-level logic, making your code more flexible and easier to maintain.

```

IVersionControl Interface
public interface IVersionControl{
    void Commit(string message);
    void Push();
    void Pull();
}

GitVersionControl Class
public class GitVersionControl : IVersionControl{
    public void Commit(string message) { Console.WriteLine($"Git commit:

```

```

{message}"); }
    public void Push() {      Console.WriteLine("Git push");  }
    public void Pull() {      Console.WriteLine("Git pull");    }
}

DevelopmentTeam Class
public class DevelopmentTeam {
    private readonly IVersionControl _versionControl;

    public DevelopmentTeam(IVersionControl versionControl) {
        _versionControl = versionControl;
    }
    public void Work()  {
        _versionControl.Commit("Initial commit");
        _versionControl.Push();
        _versionControl.Pull();
    } }

Usage Example
public class Program{
    public static void Main()  {
        IVersionControl versionControl = new GitVersionControl();
        DevelopmentTeam team = new DevelopmentTeam(versionControl);
        team.Work();
    } }

```

Exception Handling

Tuesday, February 4, 2025 5:20 PM

What is Exception Handling? Exception handling is a mechanism to handle runtime errors in your application. It ensures that the flow of the program is maintained even when unexpected situations occur.

1. Try Block: Contains the code that may cause an exception.
2. Catch Block: Handles the exception if one occurs.
3. Finally Block: Executes code after the try and catch blocks, regardless of whether an exception was thrown or not.
4. Throw Statement: Used to explicitly throw an exception.

Custom Exceptions

- You can create your own exception classes to handle specific scenarios in your application. Custom exceptions should inherit from the System.Exception class.

```
using System;
public class CustomException : Exception {
    public CustomException(string message) : base(message) { }
}

class Program{
    static void Main() {
        try { throw new CustomException("This is a custom exception."); }
        catch (CustomException ex) {
            Console.WriteLine("Caught custom exception: " + ex.Message);
        }
    }
}
```

Exception Filters

- Exception filters allow you to specify a condition for catching exceptions. This can be useful for logging or handling specific cases.

```
try { // Code that may cause an exception }
catch (Exception ex) when (ex.Message.Contains("specific condition")) {
    // Handle the exception if the condition is met
}
```

AggregateException

- When dealing with tasks and parallel programming, multiple exceptions can occur. The AggregateException class is used to handle such scenarios.

Generics

Tuesday, February 4, 2025 5:23 PM

- Generics allow you to define a class, method, delegate, or interface with a type any parameter.

Benifits of Generic:

- Type Safety: Generics provide compile-time type checking, reducing runtime errors.
- Performance: Generics eliminate the need for boxing and unboxing when working with value types, improving performance.
- Code Reusability: You can create a single class or method that works with any data type, reducing code duplication.

Generic Classes :-

```
public class GenericList<T>{  
    private T[] items;  
    private int count;  
  
    public void Add(T item) {    // Add item to the list    }  
    public T Get(int index) {    return items[index];    }  
}
```

* Generic Methods :-

```
public void Swap<T>(ref T a, ref T b) {  
    T temp = a;  
    a = b;  
    b = temp;  
}
```

* Generic Interfaces

```
public interface IRepository<T>  
{  
    void Add(T item);  
    T Get(int id);  
}
```

* Generic Delegates

```
public delegate T Transformer<T>(T arg);
```

Collections

Tuesday, February 4, 2025 5:24 PM

Generic Collections :- These collections are type-safe, meaning they store elements of a specific type.

- List<T> - A dynamic array that can grow and shrink in size.
- Dictionary<TKey, TValue> - Stores key-value pairs and allows quick lookups based on the key.
- Queue<T> - A first-in, first-out (FIFO) collection.
- Stack<T> - A last-in, first-out (LIFO) collection.
- HashSet<T> - A collection that contains no duplicate elements.

Non-Generic Collections :- These collections do not enforce type safety, meaning they can store any type of object.

- ArrayList - A dynamically sized collection that can store elements of any type.

It can hold elements of any type but requires casting when retrieving.

- Hashtable - A non-generic collection that stores key-value pairs.

Common Collection Operations

* Adding Elements - Add()

* Removing Elements -

- Remove(): Removes a specific item.
- RemoveAt(): Removes an element at a specified index (for List<T>).
- Clear(): Clears all items from the collection.

* Accessing Elements -

- [] Indexer: Access an element by its index

* Checking Membership -

- Contains(): Checks if a collection contains a certain element.

* Sorting

- Sort(): Sorts elements in ascending order

* Collection Initialization - var numbers = new List<int> { 1, 2, 3, 4 };

CSharp language enhancements

Tuesday, February 4, 2025

5:28 PM

Implicit Type Variable

- **Var**

- the var keyword is used to declare an implicitly typed local variable. This means that the type of the variable is determined by the compiler at compile time based on the type of the expression used to initialize the variable. Once the type is inferred, it cannot be changed.
- var number = 10; // The compiler infers that 'number' is of type int

- **Dynamic Variable**

- the dynamic keyword is used to declare a variable whose type is not determined until runtime. This allows for more flexibility, but it also means that type checking is deferred until the variable is actually used.
- The type of the variable can change at runtime.
- Using dynamic can lead to runtime errors if the operations performed on the variable are not valid for its actual type.

```
dynamic variable = 1; // Initially an int
variable = "Hello, Copilot!"; // Now a string
variable = new List<int> { 1, 2, 3 }; // Now a List<int>
```

- **Object Initializers**

- Collection initializers allow you to add elements to a collection at the time of creation. This is particularly useful for collections like lists, dictionaries.

```
var numbers = new List<int> { 1, 2, 3, 4, 5 };
var people = new List<Person>
{
    new Person { Name = "Alice", Age = 25 },
    new Person { Name = "Bob", Age = 28 }
};
```

- **Anonymous types**

- Anonymous types in C# provide a convenient way to encapsulate a set of read-only properties into a single object without having to explicitly define a class. They are particularly useful for creating objects on the fly, especially when working with LINQ queries.
- Read-Only Properties: The properties of an anonymous type are read-only. You cannot change their values after the object is created.
- Limited Scope: Anonymous types are typically used within the scope they are created, such as within a method.

```
var person = new { Name = "John Doe", Age = 30 };
var people = new List<Person> {
    new Person { Name = "Alice", Age = 25 },
    new Person { Name = "Bob", Age = 28 }
};

var query = from p in people
            select new { p.Name, p.Age };

foreach (var person in query) {
    Console.WriteLine($"Name: {person.Name}, Age: {person.Age}");
}
```


Delegates

Tuesday, February 4, 2025 5:35 PM

- **What is delegates**

- In C#, a delegate is a type that defines a method signature, and it can hold references to methods with that signature. Essentially, it acts like a pointer to a method, allowing you to call methods.
- A delegate can be used to pass methods as arguments, define callbacks, and handle events. You can think of it as a type-safe function pointer.

```
public delegate int MyDelegate(int x, int y);
class Program
{
    static int Add(int x, int y) { return x + y; }
    static void Main()
    {
        MyDelegate myDelegate = new MyDelegate(Add);
        int result = myDelegate(10, 20);
        Console.WriteLine(result); // Output: 30
    }
}
```

- **Needs of delegate**

- Event Handling- Delegates are used to define the signature of event handlers, enabling decoupling between event sources and handlers.
- when you subscribe to an event, you typically use a delegate to define the method signature that the event handler must have. This decouples the event source from the event handler, allowing more flexible and dynamic handling of events.
- Callback methods - Delegates allow methods to be passed as parameters
- Anonymous Methods and Lambda Expressions
- Multicast Delegates - A delegate in C# can reference more than one method, allowing for multicast operations. This means when you invoke a delegate, all the methods it references are called in the order they were added.
- Functional Programming Patterns - Delegates are often used to implement functional programming patterns

- **Single Cast Delegate**

- The Greeter class now has a constructor that accepts a greetingPrefix and initializes the _greetingPrefix field.
- When creating a Greeter instance, you provide a custom greeting prefix (e.g., "Good morning").
- The SayHello method uses that prefix along with the name to generate the greeting.
- The delegate greet points to the SayHello method, and when invoked with "Alice", it prints: Good morning, Alice!.

```
public delegate void GreetDelegate(string name);
public class Greeter{
    private string _greetingPrefix;
    public Greeter(string greetingPrefix)
    { _greetingPrefix = greetingPrefix; }
    public void SayHello(string name)
    { Console.WriteLine($"{_greetingPrefix}, {name}!"); }
}

public class Program {
    public static void Main() {
        Greeter greeter = new Greeter("Good morning");
        SayHello GreetDelegate greet = greeter.SayHello;
        greet("Alice"); // Output: Good morning, Alice!
    }
}
```

- **Multicast Delegate**

- Delegate Definition: GreetDelegate is a delegate type that can point to methods that take a string argument and return void.
- Methods: The Greeter class contains two instance methods:
SayHello: Prints a greeting message.
SayGoodbye: Prints a farewell message.
- Multicast Delegate:
 - The greet delegate is initially assigned to the SayHello method.
 - We use += to add the SayGoodbye method to the delegate, creating a multicast delegate.
- Invocation: When greet("Alice") is called, both methods (SayHello and SayGoodbye) are invoked in the order they were added, printing the greetings and goodbyes.
- Removing Methods:
 - If you want to remove a method from the multicast delegate, you can use -=.
greet -= greeter.SayGoodbye;

```
public delegate void GreetDelegate(string name);
public class Greeter
{
    private string _greetingPrefix;
    public Greeter(string greetingPrefix)
    { _greetingPrefix = greetingPrefix; }

    public void SayHello(string name)
    { Console.WriteLine($"{_greetingPrefix}, {name}!"); }
    public void SayGoodbye(string name)
    { Console.WriteLine($"Goodbye, {name}!"); }
}

public class Program
{
    public static void Main() {
        Greeter greeter = new Greeter("Good morning");
        GreetDelegate greet = greeter.SayHello;
        greet += greeter.SayGoodbye;
        greet("Alice");
    }
}
```

Delegates Enhancements

Tuesday, February 4, 2025 5:42 PM

Lambda Expression

- A lambda expression is a shorthand for defining anonymous methods. It allows you to define a method inline without needing the delegate keyword. Lambda expressions are commonly used with delegates, LINQ queries, and event handlers.

```
public delegate int MathOperation(int a, int b);
public static void Main()
{
    MathOperation multiply = (a, b) => a * b;
    Console.WriteLine(multiply(4, 5)); // Output: 20
}
```

- **Func**
 - Func is a built-in generic delegate type that represents methods that return a value. It can be used with lambda expressions, anonymous methods, or named methods to handle functions that take arguments and return a result.
 - Func<T1, T2, ..., TResult> is a delegate type.
 - It represents methods that take parameters (T1, T2, etc.) and return a value of type TResult.
 - The number of input parameters (T1, T2, ...) can range from 0 to 16, but the last parameter is always the return type.

```
class Program
{
    static void Main()
    {
        Func<int, int, int> add = (a, b) => a + b;
        Console.WriteLine(add(3, 4)); // Output: 7
    }
}
```

- **Action**
 - Action is another built-in delegate type, similar to Func, but it represents methods that do not return a value. It is used for methods that perform actions but do not produce a result (i.e., methods that return void).
 - A method that takes parameters upto 16(T1, T2, ...) but returns void.

```
class Program
{
    static void Main()
    {
        Action<int, int> addAndPrint = (a, b) => Console.WriteLine($"Sum: {a + b}");
        addAndPrint(5, 10); // Output: Sum: 15
    }
}
```

- Predicate
- This lambda expression accepts only one parameter and returns a Boolean value. By default, the result that is inferred from the expression is a Boolean value.

```
static void Main(string[] args)
{
    Predicate<Product> checkPrice = x => x.Price > 0;
    Product product = new Product("Tennis Racket", 1500, 15, 1);
    Console.WriteLine(checkPrice(product));
}
```

Extension Methods

Tuesday, February 4, 2025 5:45 PM

- Extension methods allow you to add new functionality to existing types without modifying their source code. This is useful when you want to enhance a class or interface, such as adding utility methods, without directly altering the class.
- Extension methods must be defined in static classes.
- They must be static methods.
- The first parameter of an extension method specifies the type it extends, and it is preceded by the `this` keyword.

```
public static class ListExtensions
{
    public static List<int> GetEvenNumbers(this List<int> list)
    {
        return list.FindAll(n => n % 2 == 0);
    }
}

static void Main() {
    List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    var evenNumbers = numbers.GetEvenNumbers();
}
```

Asynchronous programming

Tuesday, February 4, 2025 5:45 PM

- **What is Asynchronous programming**

- Asynchronous programming in C# allows you to run tasks concurrently without blocking the main thread, which is particularly useful in I/O-bound operations (like reading files, querying databases, or making web requests).
- **Async** Used to define a method as asynchronous. An async method must return either Task, Task<T>, or void (though void is typically used only for event handlers).
- **Await** Used to await the completion of an asynchronous operation. It ensures the method continues executing once the awaited task completes.
- **Task.Run** is often used to run CPU-bound tasks asynchronously on a separate thread pool thread.

```
await Task.Run(() => function());
```

- **Task.Delay** is used to create a non-blocking delay (similar to Thread.Sleep but without blocking the thread).
- **Task.WhenAll** is used to wait for multiple asynchronous tasks to complete. It is useful when you want to run multiple tasks concurrently and wait for all of them to finish.

```
Task task1 = Task.Delay(2000); // Simulate a delay of 2 seconds
Task task2 = Task.Delay(3000); // Simulate a delay of 3 seconds
await Task.WhenAll(task1, task2);
```

- **Task.WhenAny** is used when you want to continue as soon as any one of a group of tasks completes. This is useful for scenarios where you're waiting for the first task to finish (such as handling timeouts or canceling requests).
- **ConfigureAwait(false)** is used to avoid attempting to resume the task on the original synchronization context.

LINQ

Tuesday, February 4, 2025 5:50 PM

Events

Tuesday, February 4, 2025 5:53 PM

What are Events?

- **Events** are a way to provide notifications or signals when something occurs in your program. An event is typically used to implement the **observer pattern**, where one object (the *publisher*) triggers an event, and other objects (the *subscribers*) handle that event.
- An event allows an object to notify other objects when something important happens, without requiring the objects to know about each other directly.

Key Concepts

- **Publisher:** The class that raises the event. It triggers the event when a specific action happens.
- **Subscriber:** The class that listens for or handles the event when it is triggered by the publisher.
- **Event Handler:** A method that gets invoked when the event is raised.

Defining an Event

- An event is defined using the event keyword. Events are typically based on delegates, specifically **multicast delegates**, which can point to multiple methods.

```
public delegate void MyEventHandler(object sender, EventArgs e); // Delegate definition

public event MyEventHandler MyEvent; // Event definition
```

- The delegate defines the signature of the methods that can subscribe to the event.
- The event keyword ensures that the event can only be invoked from within the class that defines it.

Raising an Event

- To trigger or **raise** an event, you invoke the event in the method where the event occurs.

```
protected virtual void OnMyEvent(EventArgs e)
{
    MyEvent?.Invoke(this, e); // Null-conditional operator ensures the event has subscribers
}
```

- `MyEvent?.Invoke(this, e)` raises the event, passing the sender (`this`) and event arguments (`e`).
- The `protected virtual` allows derived classes to override the method if necessary.

Subscribing to an Event

- Other classes can subscribe to the event using the `+=` operator and provide a method that matches the delegate signature.

```
MyEventHandler handler = new MyEventHandler(MyEventHandlerMethod); // Create a delegate
MyEvent += handler; // Subscribe to the event

public void MyEventHandlerMethod(object sender, EventArgs e)
{
    Console.WriteLine("Event raised!");
}

MyEvent -= handler; // Unsubscribe from the event
```

What is EventArgs?

- `EventArgs` is a class in the `System` namespace, and it is used as the base class for event data in C#. When

you declare an event, the event handler usually receives an instance of EventArgs or a subclass of EventArgs to carry any necessary data related to the event.

- The EventArgs class itself is empty, which means it doesn't contain any properties or data. But by creating a subclass of EventArgs, you can add the relevant data (parameters) for your event.

Example of complete event

```
using System;

public class Publisher
{
    // Define the delegate
    public delegate void MyEventHandler(object sender, EventArgs e);

    // Define the event based on the delegate
    public event MyEventHandler MyEvent;

    // Method to raise the event
    protected virtual void OnMyEvent(EventArgs e)
    {
        MyEvent?.Invoke(this, e); // Null-conditional operator to avoid invoking if no subscribers
    }

    // Method to trigger the event
    public void TriggerEvent()
    {
        Console.WriteLine("Event is about to be triggered...");
        OnMyEvent(EventArgs.Empty); // Raise the event
    }
}

public class Subscriber
{
    // Event handler method that matches the delegate signature
    public void MyEventHandler(object sender, EventArgs e)
    {
        Console.WriteLine("Event raised and handled!");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Publisher publisher = new Publisher();
        Subscriber subscriber = new Subscriber();

        // Subscribe to the event
        publisher.MyEvent += subscriber.MyEventHandler;

        // Trigger the event
        publisher.TriggerEvent();

        // Unsubscribe from the event
    }
}
```

```

        publisher.MyEvent -= subscriber.MyEventHandler;
    }
}

```

- events are often associated with parameters that can carry information when the event is triggered. Here's how you can define and use events with parameters in C#.

```

using System;

public class UserEventArgs : EventArgs
{
    public string Username { get; set; }
    public string Email { get; set; }
    public string SignupMethod { get; set; }
}

public class UserService
{
    // Declare the event
    public event EventHandler<UserEventArgs> UserSignedUp;

    public void SignUp(string username, string email, string signupMethod)
    {
        // Raise the event
        OnUserSignedUp(new UserEventArgs
        {
            Username = username,
            Email = email,
            SignupMethod = signupMethod
        });
    }

    protected virtual void OnUserSignedUp(UserEventArgs e)
    {
        UserSignedUp?.Invoke(this, e);
    }
}

class Program
{
    static void Main()
    {
        UserService userService = new UserService();
        userService.UserSignedUp += (sender, e) =>
        {
            Console.WriteLine($"User signed up! Username: {e.Username}, Email: {e.Email}, Method: {e.SignupMethod}");
        };

        userService.SignUp("john_doe", "johndoe@example.com", "email");
    }
}

```

DomainEvents

Thursday, February 6, 2025 6:01 PM

- A **Domain Event** is a concept from **Domain-Driven Design (DDD)**
- A domain event is a way to express that something meaningful has occurred in the domain that other parts of the system might need to respond to.
- Business Meaningful Events
A domain event represents a business-related fact. For example, when a customer places an order, that might trigger several other actions within the system, such as notifying the warehouse, updating inventory, or sending an email. This "Order Placed" event carries the meaning of a significant business action.
- Immutability
Once a domain event has been created, it is immutable, meaning it cannot be changed. For example, when an order is placed, the event captures the OrderId, CustomerName, and DatePlaced. This data will never change.

How Domain Events Fit Into the Larger System

To understand the role of domain events better, let's look at them in context with other components like **Aggregates** and **Repositories** in Domain-Driven Design.

1. Aggregates

- In DDD, an **aggregate** is a cluster of domain objects that are treated as a single unit. The aggregate root ensures the integrity of the entire group. For example, the Order entity might be an aggregate in a system.
- When an aggregate (like Order) changes state in a way that other parts of the system should know about, it can publish a **domain event** (like OrderPlacedEvent).

2. Repositories

- A **repository** is a pattern used to abstract the data layer. Repositories manage the fetching and storing of aggregates.
- After an aggregate (like Order) triggers a domain event, a repository might persist the state of the aggregate (saving it to a database) and/or raise events.

3. Event Handlers

- **Event handlers** are responsible for reacting to domain events. These handlers might trigger additional processes, such as sending out emails, calling external services, or updating other parts of the system asynchronously.

4. Event Bus (Optional)

- In some designs, a **message bus** or **event bus** is used to route events to interested handlers. The event bus is a mechanism to publish events and allow components to subscribe and react accordingly.